Design-Time Business Process Compliance Checking with Large Language Models

Yingchao Huo, Xiaoqing Chen, Jiahao Ma, Guosheng Kang✉, Wen Li, Jiayan Xiang
[1]Hunan Provincial Key Lab. for Services Computing and Novel Software Technology, HNUST, Xiangtan, Hunan, China
[2]School of Computer Science and Engineering, Hunan University of Science and Technology, Xiangtan, China
Email: {huoyingchao666, xiaoqingchen696, jiahaoma2627, guoshengkang, wenli0023cs, jiayanxiang02}@gmail.com

***Abstract*—Design-time business process compliance (BPC) checking aims to identify potential violations before process execution. Existing approaches often rely on manually designed linguistic rules and syntactic patterns to transform natural-language compliance requirements into machine-processable representations. This paper investigates large language model (LLM)-based semantic extraction for design-time BPC checking. Natural-language compliance requirements are transformed into structured Rule Records containing modality, actor, action, condition, constraint, and exception information. We compare Direct-LLM with a reconstructed Rules-Only approach under a common semantic representation and evaluation setting. We further analyze the effects of prompt configurations and examine how the resulting Rule Records influence downstream compliance checking.**

***Index Terms*—Business Process Compliance, Design-Time Compliance Checking, Large Language Models, Compliance Requirement Parsing, Violation Detection**

I. Introduction

Business process compliance (BPC) aims to ensure that business processes conform to applicable laws, regulations, standards, and organizational policies [1]. Design-time BPC checking identifies potential violations before process execution, but a major challenge is the semantic gap between natural-language compliance requirements and formal process models. Many existing approaches rely on compliance constraints that have already been formalized, making the transformation of regulatory text into machine-processable representations an important and often labor-intensive step [1, 2].

To reduce this effort, Winter et al. [3] investigated compliance assessment directly from regulatory documents. Sun et al. [4] further proposed a design-time BPC framework that extracts fine-grained regulatory semantics, including modality, actor, action, condition, constraint, and exception, and matches them with process-model information for violation detection. Their semantic extraction combines syntactic analysis and manually designed linguistic rules. While explicit and interpretable, such extraction depends on predefined patterns and linguistic resources.

Recent work has explored Large Language Models (LLMs) for legal requirement modeling, structured regulatory-rule extraction, and BPC-related analysis [5–8]. Although some of these studies adopt BPC-relevant semantics or extract fine-grained regulatory information, their primary objectives differ from directly comparing LLM-based and rule-based extraction of individual compliance requirements within a common design-time BPC setting. It therefore remains useful to examine how Direct-LLM extraction behaves under the same target representation and evaluation setting as a reconstructed non-LLM baseline, and how the resulting Rule Records affect downstream BPMN compliance checking.

In this paper, we investigate Direct-LLM semantic extraction within this design-time BPC setting. Natural-language compliance requirements are transformed into structured Rule Records, while BPMN process models are converted into structured process information for downstream matching and violation analysis. We compare Direct-LLM with an independent Rules-Only reconstruction under common inputs, semantic representations, reference annotations, and evaluation procedures. We further analyze the effects of different prompt configurations and examine how the resulting Rule Records influence downstream compliance checking.

The main contributions of this work are:

- We apply Direct-LLM extraction to an existing fine-grained BPC semantic representation and generate structured Rule Records for downstream compliance checking.
- We conduct a controlled comparison between Rules-Only and Direct-LLM extraction, analyze their strengths and limitations at the semantic-element level, and examine the effects of different prompt configurations.
- We connect the extracted Rule Records with downstream compliance checking and evaluate their effects on the three original violation types: missing action, incorrect actor, and out-of-order execution.

The remainder of this paper is organized as follows. Section II reviews related work. Section III presents the problem formulation and framework. Section IV describes the methodology. Section V reports the experiments and results, and Section VI concludes the paper.

II. Related Work

Design-time business process compliance checking has been studied using logic-based, pattern-based, and other formal compliance representations [1, 2]. These approaches provide precise and machine-processable constraints, but regulatory requirements generally need to be formalized before automated checking can be performed. To reduce this manual effort, Winter et al. [3] investigated compliance assessment directly from regulatory documents. Sun et al. [4] further proposed a design-time BPC approach that extracts fine-grained semantic

information from regulatory text and combines it with process-model information for violation detection.

Recent work has explored large language models (LLMs) for legal and regulatory modeling. De Kinderen and Winter [5] studied prompt-based generation of legal requirement models. Barrientos et al. [6, 7] investigated regulatory-change detection and its relevance or impact on business process compliance, including BPC-relevant semantics related to the representation of Sun et al. [4]. Guliani et al. [8] proposed De Jure, an LLM-based pipeline for fine-grained structured extraction of regulatory rules with iterative evaluation and repair. These studies demonstrate the potential of LLMs for regulatory analysis, but their primary objectives differ from directly extracting individual compliance requirements under a shared Sun-style BPC representation and comparing the resulting Rule Records in the same design-time BPMN compliance-checking pipeline. In contrast, this work conducts a controlled comparison between a reconstructed non-LLM extractor and Direct-LLM under the same target representation and evaluator, and further examines their downstream compliance-checking outcomes.

III. Problem Formulation and Framework

A. Problem Formulation

Following the design-time BPC setting of Sun et al. [4], we consider a BPMN process model P and a natural-language compliance requirement q. The process model is transformed into a Process Record $\mathcal{P}$, which contains process elements, participants or actors, control-flow relations, and label semantics. The compliance requirement is transformed into a Rule Record $\mathcal{R}$, which contains modality, actor, action, condition, constraint, exception, and supporting evidence. Design-time compliance checking then uses $(\mathcal{P}, \mathcal{R})$ for rule–process matching and violation analysis.

B. Overall Framework

As illustrated in Fig. 1, the proposed framework consists of three stages: Process Model Parsing, Compliance Requirement Parsing, and Compliance Checking. Stage 1 parses the BPMN process model and organizes the extracted process elements, participants, control-flow relations, and label semantics into a structured Process Record. Independently, Stage 2 uses Direct-LLM semantic extraction to transform the natural-language compliance requirement into a structured Rule Record containing fine-grained compliance semantics. Stage 3 combines the Process Record and Rule Record for rule–process matching and detects missing-action, incorrect-actor, and out-of-order violations. The detailed Direct-LLM procedure in Stage 2 is presented in Section IV-B.

IV. Methodology

A. Process Model Parsing

Following Sun et al. [4], Stage 1 uses a deterministic BPMN parser to extract process elements, participant information, sequence flows, and textual labels. Sequence flows are further used to derive control-flow edges, transitive reachability, and activity-order relations. Actor, action, and business-object semantics are obtained from process labels using a fixed linguistic analysis procedure following the process-label analysis strategy of Sun et al. The extracted information is organized into a structured Process Record containing activities, participants, control-flow relations, and label semantics for downstream compliance checking.

B. Compliance Requirement Parsing

Stage 2 transforms each natural-language compliance requirement into a structured Rule Record. Following the semantic representation of Sun et al. [4], each record is organized into clauses containing modality, actor, action, condition, constraint, and exception information. Span-bearing elements are represented by source-anchored evidence spans with stable identifiers and normalized values for downstream matching. Character offsets use zero-based start and exclusive end positions, and the recorded evidence text must correspond exactly to the associated source span. Actor–action associations reference the identifiers of extracted actors and actions, while ordering relations reference the identifiers of their before/after action endpoints.

Prompt Design. As shown in Fig. 2, the prompt consists of a shared base layer and three configurable components. The shared base layer defines the extraction task, target Rule Record schema, source and evidence representation, span and identifier requirements, and the basic structural output interface. It is retained throughout all prompt configurations. On top of this base layer, E provides synthetic worked examples, S provides detailed semantic interpretation guidance, and J provides additional JSON and output-discipline instructions. Only E, S, and J are varied in the factorial experiment, while the shared base layer remains fixed.

Direct-LLM Semantic Extraction. Given the requirement and prompt, Direct-LLM directly generates structured semantic elements, supporting evidence, and relations without using predictions from the rule-based baseline or reference annotations. The generated information is organized into a candidate Rule Record under a unified JSON representation. This design allows semantic extraction to be performed within a single LLM call while preserving the fine-grained representation required by downstream compliance checking.

Output Validation and Normalization. The generated candidate record is processed by a deterministic validation and normalization procedure before being used in Stage 3. The procedure checks schema conformance, exact correspondence between evidence text and source offsets, containment of element spans within their clauses, uniqueness of element identifiers, and referential consistency of actor–action and ordering relations. If an evidence span is misaligned, it is re-anchored only when the reported text has a unique exact occurrence in the corresponding source clause; otherwise, the unrecoverable element is removed. Normalized values are retained for downstream semantic matching, while the

original evidence text and source offsets remain unchanged. This procedure repairs structural and alignment errors without introducing new semantic interpretations.

C. Compliance Checking

Stage 3 combines the Process Record and Rule Record for rule–process matching and violation detection. Following Sun et al. [4], we retain the three original violation types: missing action, incorrect actor, and out-of-order execution. Let A_r and U_r denote the actions and actors contained in rule r, respectively, and let $\mathrm{sim}(x_r, x_p)$ denote the semantic similarity between a rule-side element and a process-side element. Thresholds τ, γ, and θ are used for rule–process matching, action mapping, and actor comparison, respectively.

Rule–Process Matching. Each rule action is matched to its most similar process-side action, forming the candidate mapping set $D_{r,p}$. Rule actors are matched analogously, forming $O_{r,p}$. The action and actor matching ratios and the overall rule–process matching score are defined as

$$\begin{aligned} m_A(r,m,\tau) &= \frac{\sum_{(a_r,a_m)\in D_{r,m}} sim(a_r,a_m) > \tau}{|D_{r,m}|}, \\ m_O(r,m,\tau) &= \frac{\sum_{(o_r,o_m)\in O_{r,m}} sim(o_r,o_m) > \tau}{|O_{r,m}|}, \\ \mathcal{F}(r,m,\tau) &= \max\{m_A(r,m,\tau), m_O(r,m,\tau)\}. \end{aligned} \tag{1}$$

When $A_r = \emptyset$, $m_A(r,p;\tau)$ is defined as 0; analogously, $m_U(r,p;\tau) = 0$ when $U_r = \emptyset$. The resulting mappings provide the basis for the three downstream violation checks.

Missing-Action Violation. For each required rule action $a_r \in A_r$, let $s(a_r,p) = \max_{a_p\in A_p} \mathrm{sim}(a_r,a_p)$, with $s(a_r,p) = 0$ when $A_p = \emptyset$. For $|A_r| > 0$, the missing-action score is

$$B_{r,m,\gamma} = \{(a_r,a_m) \in D_{r,m} \mid sim(a_r,a_m) < \gamma\}. \tag{2}$$

$$action_violation(r,m,\gamma) = \frac{|B_{r,m,\gamma}|}{|A_r|}. \tag{3}$$

Thus, a rule action with no process-side candidate is counted as missing. If $A_r = \emptyset$, the case is unevaluable for missing-action checking and no decision is produced.

Incorrect-Actor Violation. Actor checking is performed after the corresponding rule action has been successfully mapped to a process activity. Let $R_{r,p,\gamma}$ denote the rule actors associated with successfully mapped actions and $C_{r,p,\gamma}$ the union of actor or business-object candidates bound to those matched activities. Following the Sun-style definition, for $|R_{r,p,\gamma}| > 0$ and $C_{r,p,\gamma} \neq \emptyset$,

$$\begin{aligned} R_{r,m,\gamma} = \big\{ & g \in P_r \mid \exists (a_r,a_m) \in D_{r,m}, \\ & f_r(g) = a_r \wedge sim(a_r,a_m) > \gamma \big\}. \end{aligned} \tag{4}$$

$$\begin{aligned} C_{r,m,\gamma} = \big\{ & r \in bs_obj(A_m \cup E_m) \cup R_m \mid \\ & \exists g \in R_{r,m,\gamma}, \\ & sim(f_r(g), f_m(r)) > \gamma \big\}. \end{aligned} \tag{5}$$

$$\begin{aligned} I_{r,m,\gamma,\vartheta} = \big\{ & r \in R_{r,m,\gamma} \mid \exists r' \in C_{r,m,\gamma}, \\ & sim(r,r') < \vartheta \big\}. \end{aligned} \tag{6}$$

$$actor_violation(r,m,\gamma,\vartheta) = \frac{|I_{r,m,\gamma,\vartheta}|}{|R_{r,m,\gamma}|}. \tag{7}$$

For non-empty $C_{r,p,\gamma}$, this minimum test is equivalent to the existential condition $\exists u_p \in C_{r,p,\gamma} : \mathrm{sim}(u_r,u_p) < \theta$ in Sun

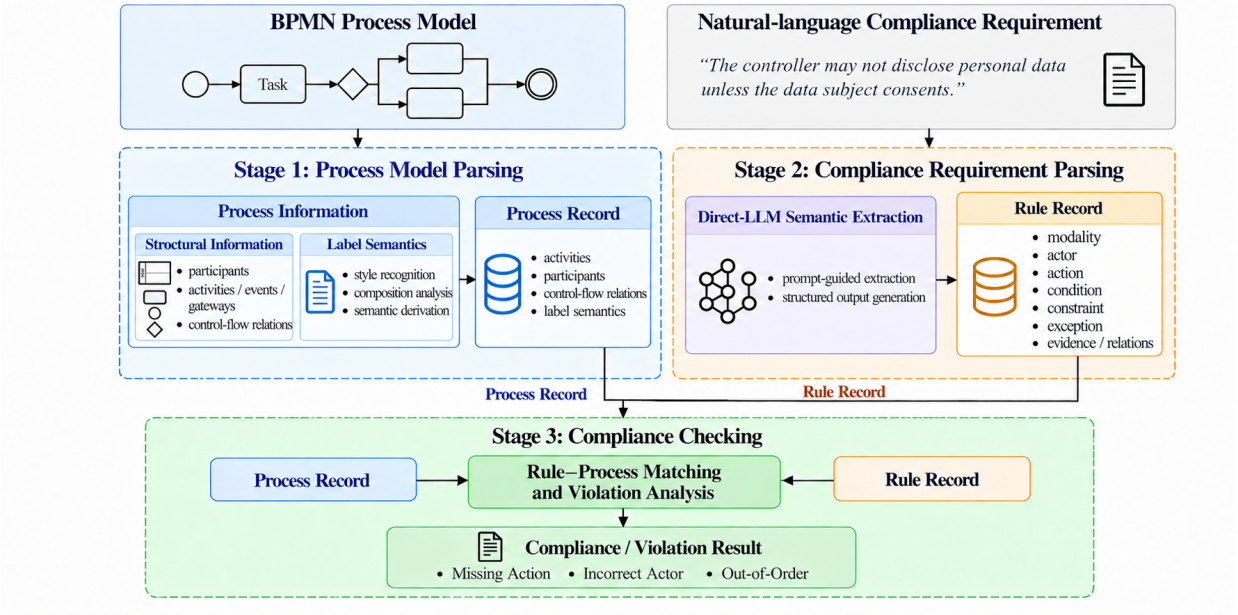


Fig. 1: Overall framework for design-time business process compliance checking.

et al. [4]; the minimum form is used as an implementation-equivalent rewriting. If the required actor, actor–action association, or corresponding process-side candidate is unavailable, the case is unevaluable for actor checking and no decision is produced.

Out-of-Order Violation. Ordering checks are performed for rule-side ordering relations whose endpoints can both be mapped to process activities. Let $U_{r,p,\gamma}$ denote the set of evaluable ordering relations and $S_{r,p,\gamma} \subseteq U_{r,p,\gamma}$ the subset satisfied by the process control flow. For $|U_{r,p,\gamma}| > 0$, the out-of-order score is

$$U_{r,m,\gamma} = \{(u_r, u'_r) \in U_r \mid \exists\{(u_r, u_m) \in D_{r,m}, (u'_r, u'_m) \in D_{r,m}\}, \; sim(u_r, u_m) > \gamma \wedge sim(u'_r, u'_m) > \gamma\}. \quad (8)$$

$$S_{r,m,\gamma} = \{(u_r, u'_r) \in U_{r,m,\gamma} \mid \exists\{(u_r, u_m) \in D_{r,m}, (u'_r, u'_m) \in D_{r,m}\}, \; (u_m, u'_m) \in F_m \wedge (u'_m, u_m) \notin F_m\}. \quad (9)$$

$$order_violation(r, m, \gamma) = \frac{|U_{r,m,\gamma}| - |S_{r,m,\gamma}|}{|U_{r,m,\gamma}|}. \quad (10)$$

An ordering relation is satisfied when the first mapped activity can reach the second activity in the process control flow while reverse reachability does not hold. If $|U_{r,p,\gamma}| = 0$, the case is treated as unevaluable for out-of-order checking and no order-violation decision is produced.

V. Experiments and Results

We evaluate the proposed approach from three perspectives. First, we compare Rules-Only and Direct-LLM for fine-grained semantic extraction. Second, we analyze how different prompt components affect Direct-LLM through a full factorial experiment. Finally, we examine whether the extracted Rule Records lead to different downstream compliance-checking performance.

A. Experimental Setup

Stage 2 evaluation. We use an independently reconstructed EStG-150 benchmark containing 150 compliance requirements. Its reference annotations form an LLM-assisted, human-adjudicated Gold standard constructed for this study rather than the original unpublished annotations of Sun et al. [4]. Rules-Only and Direct-LLM are evaluated against the same frozen Gold and evaluator. For actor, action, condition, constraint, and exception, a prediction matches a Gold span when their character intervals have a non-empty overlap; Overall Micro-F1 pools TP, FP, and FN over these five fields. Modality is evaluated separately by four-class Macro-F1 and is excluded from the overall score.

Rules-Only baseline. Rules-Only is an independent reconstruction of the Stage 2 approach described by Sun et al. [4], based on syntactic analysis and manually designed linguistic rules. It is treated as a reconstructed baseline rather than the original implementation. Both methods use the same frozen inputs, Gold annotations, target representation, and evaluator.

Direct-LLM and prompt configurations. All Direct-LLM experiments use DeepSeek-V4-Pro-0813 with temperature 0, top-p 1.0, and a maximum output length of 4096 tokens. Table I and Table II are separate frozen runs; the provider exposes no supported random seed, so all differences are interpreted descriptively without statistical-significance claims. The shared prompt base remains fixed, while E, S, and J denote semantic examples, semantic guidance, and additional output discipline. Configuration 111 is the intact frozen full prompt, the other configurations are controlled factorial variants, and 000 retains the shared base rather than representing an unprompted LLM.

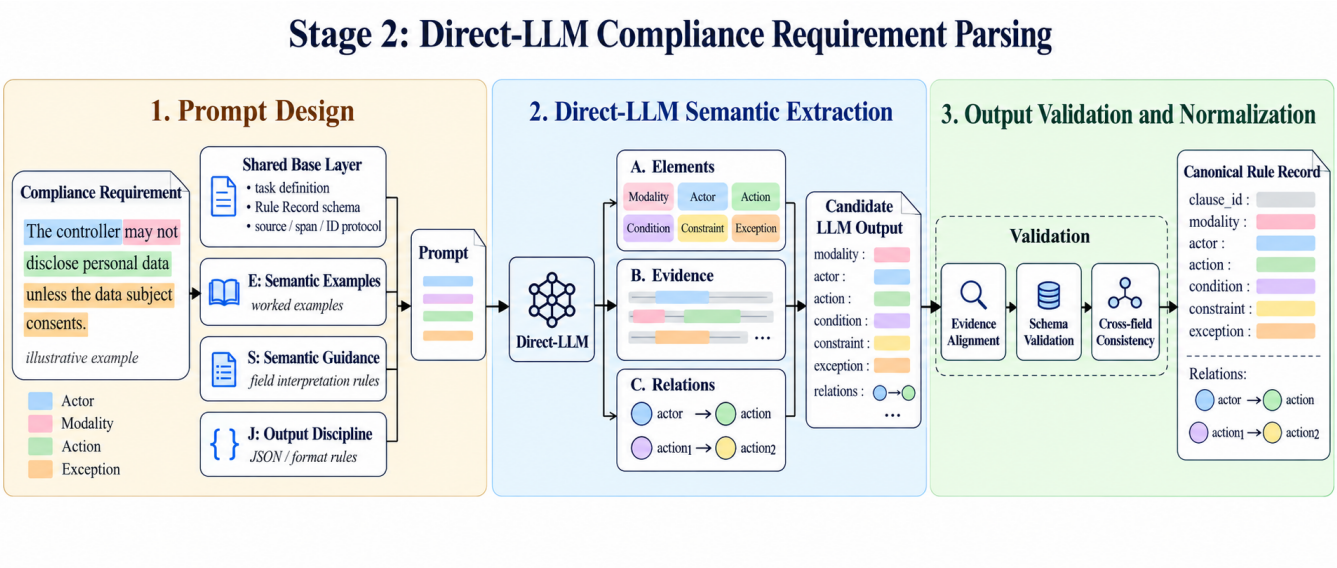


Fig. 2: Detailed workflow of Stage 2.

TABLE I: Semantic extraction performance of Rules-Only and Direct-LLM.

Element	Rules-Only	Direct-LLM
Modality (Macro-F1)	0.713	**0.769**
Actor	**0.820**	0.758
Action	0.893	**0.944**
Condition	0.774	**0.838**
Constraint	0.618	**0.743**
Exception	**0.880**	0.762
Overall (Micro-F1)*	0.763	**0.838**

*Pooled over actor, action, condition, constraint, and exception; modality is excluded.

Stage 3 evaluation. Stage 3 uses a separately constructed GDPR-based benchmark because downstream checking requires paired compliance requirements and BPMN models. Nine GDPR rule texts are segmented into 74 sentences and paired with 115 case records, yielding 201 applicable case–type cells per method: 113 missing-action, 80 incorrect-actor, and 8 out-of-order cells. These cells are fixed independently of method outputs; an unevaluable prediction remains in the evaluation, counting as FN for a violated reference and as unknown rather than TN for a satisfied reference. Overall precision, recall, and F1 in Table III pool TP, FP, and FN over these cells. After development-stage calibration using a shared objective across both methods, Sentence-BERT `all-mpnet-base-v2` was selected with $\tau = 0.8$, $\gamma = 0.55$, and $\theta = 0.45$. Because the same benchmark was used during development and calibration, the results are descriptive rather than an independently held-out test.

B. Semantic Parsing Results

Table I shows that the two approaches have different field-level strengths. Direct-LLM achieves higher F1 for action, condition, and constraint, and higher modality macro-F1, whereas Rules-Only performs better for actor and exception. The largest Direct-LLM gain occurs for constraint extraction, increasing from 0.618 to 0.743. Overall pooled F1 over the five span-bearing fields increases from 0.763 to 0.838, an absolute improvement of approximately 7.5 percentage points.

The results therefore do not indicate uniform superiority across all semantic elements. Rules-Only remains stronger for actor and exception extraction, while Direct-LLM performs better on most of the remaining semantic fields. These field-level results characterize the extraction behavior of the two methods on EStG-150; the downstream experiment in Section V-D is evaluated separately on the GDPR-based benchmark.

C. Prompt Factorial Analysis

Table II shows a non-additive interaction among the prompt components. Enabling E improves F1 in all four matched comparisons, making semantic examples the most consistently beneficial component. In contrast, S provides substantial gains when examples are absent but slightly decreases F1 when E is already present, suggesting partial overlap between examples and detailed semantic guidance. The effect of J is comparatively small and inconsistent. This does not imply that structured output requirements are unnecessary, because the shared schema and basic output constraints remain active in every configuration.

TABLE II: Factorial analysis of Direct-LLM prompt components.

E	S	J	F1	ΔF1 vs. 111
0	0	0	0.7380	-0.0844
0	0	1	0.7261	-0.0963
0	1	0	0.8107	-0.0117
0	1	1	0.8142	-0.0082
1	0	0	**0.8354**	**+0.0130**
1	0	1	0.8299	+0.0075
1	1	0	0.8268	+0.0044
1	1	1	0.8224	0.0000

Configuration 100 achieves the highest observed F1 of 0.8354, whereas the intact full-prompt configuration 111 obtains 0.8224. Thus, increasing prompt detail does not monotonically improve extraction performance. Because the full prompt was frozen before the factorial experiment, configuration 100 is treated as an ablation observation rather than a post-hoc replacement of the main method.

TABLE III: Performance comparison of downstream compliance checking.

Method	Precision	Recall	F1
Sun-style Rules-Only	0.3814	0.5362	0.4458
Direct-LLM	**0.4393**	**0.6812**	**0.5341**

D. Downstream Compliance Checking

Rules-Only and Direct-LLM are applied to the same GDPR compliance texts, and their resulting Rule Records are processed by the same frozen Stage 3 pipeline. Table III reports pooled performance over the 201 applicable evaluation cells. Direct-LLM achieves an overall F1 of 0.5341 compared with 0.4458 for Rules-Only, an absolute gain of 8.83 percentage points. Recall increases from 0.5362 to 0.6812 and precision from 0.3814 to 0.4393. At the violation-type level, missing-action F1 increases from 0.4554 to 0.4935 and incorrect-actor F1 from 0.4262 to 0.5684, while both methods obtain 0.5000 for out-of-order checking.

These downstream scores should not be directly compared with the Stage 2 extraction scores because the two experiments use different benchmarks, tasks, and evaluation units. Downstream checking additionally requires the extracted compliance semantics to be aligned with BPMN activity labels and process-control information. A post-hoc analysis of the frozen missing-action results identifies 50 of 70 false-positive method–case cells (71.4%) as mapping-related and 20 (28.6%) as broadly scope-related. Only 11 of the latter satisfy the stricter criterion for safely removable scope errors. Since this analysis concerns missing-action false positives only, it should not be interpreted as a decomposition of all Stage 3 errors. The

results nevertheless identify rule–process semantic alignment as a major remaining limitation.

VI. Conclusion

This paper investigates Direct-LLM semantic extraction within a three-stage design-time BPC framework. Rules-Only and Direct-LLM produce the same structured Rule Record representation, enabling controlled comparison in Stage 2 and downstream evaluation under the same Stage 3 pipeline. On EStG-150, Direct-LLM performs better for modality, action, condition, and constraint, while Rules-Only remains stronger for actor and exception; Overall Micro-F1 over the five span-bearing fields increases from 0.763 to 0.838. The factorial analysis further shows that semantic examples provide the most consistent prompt benefit, while detailed semantic guidance and additional output discipline exhibit non-additive effects.

On the separate GDPR-based development benchmark, Direct-LLM Rule Records yield a downstream F1 of 0.5341 compared with 0.4458 for Rules-Only under the same checking pipeline. These results should not be interpreted as a direct field-level transfer of the EStG-150 results. The downstream analysis identifies rule–process semantic alignment, particularly the alignment between extracted compliance actions and BPMN activity labels, as an important remaining challenge. Future work will therefore focus on improving this alignment and evaluating the approach on independently held-out benchmarks.

Acknowledgment

This work is partially supported by the Hunan Provincial Natural Science Foundation of China under Grant No: 2026JJ50519. *Guosheng Kang* is the corresponding author.

References

[1] M. Hashmi, G. Governatori, H.-P. Lam, and M. T. Wynn, "Are we done with business process compliance: state of the art and challenges ahead," *Knowledge and Information Systems*, vol. 57, no. 1, pp. 79–133, 2018.

[2] A. Zasada, M. Hashmi, M. Fellmann, and D. Knuplesch, "Evaluation of compliance rule languages for modelling regulatory compliance requirements," *Software*, vol. 2, no. 1, pp. 71–120, 2023.

[3] K. Winter, H. van der Aa, S. Rinderle-Ma, and M. Weidlich, "Assessing the compliance of business process models with regulatory documents," in *Conceptual Modeling*. Springer, 2020, pp. 189–203.

[4] X. Sun, S. Yang, C. Zhao, and D. Yu, "Design-time business process compliance assessment based on multi-granularity semantic information," *The Journal of Supercomputing*, vol. 80, no. 4, pp. 4943–4971, 2024.

[5] S. de Kinderen and K. Winter, "Towards taming large language models with prompt templates for legal grl modeling," in *Enterprise, Business-Process and Information Systems Modeling*. Springer, 2024, pp. 213–228.

[6] M. Barrientos, K. Winter, and S. Rinderle-Ma, "Impact analysis of regulatory requirement changes on business process compliance," *Information and Software Technology*, vol. 194, p. 108079, 2026.

[7] M. Barrientos, J. Loebbecke, K. Winter, and S. Rinderle-Ma, "Taming the complexity of legal change for business process compliance," *Business & Information Systems Engineering*, pp. 1–26, 2026.

[8] K. Guliani, D. Gill, D. Landsman, N. Eshraghi, K. Kumar, and L. Gondara, "De jure: Iterative llm self-refinement for structured extraction of regulatory rules," *arXiv preprint arXiv:2604.02276*, 2026.